The next step is bringing the cache out of one application, in its own process, and communicate with it through a certain protocol: this can be HTTP, Thrift or even RPC.

A typical case are the several layers of cache in a web application:

- A CDN to cache and deliver static (and some dynamic content).
- A web cache that caches content from the web server (usually entire pages, might be the CDN itself).
- An application cache storing the result of expensive server computations;
- A DB cache, storing the most used rows, tables or pages from a database.

Finally, for large applications with thousands of requests per second (or more), caches need to be scaled together with the rest of the web stack. This means that a single process might not be enough anymore to hold all the entries that the application needs to cache, in order to run smoothly (for instance, if we have a million access per second to a database, a cache in front of it that can only store a million elements is probably close to useless, as there would be too high a turnaround, and all requests would basically go ahead to the DB, crushing it to a halt in a few minutes).

For this reason, distributed caches have been introduced. These caches, like for example Cassandra, use several processes, called nodes, each one of them being a standalone cache, plus another process (sometimes a special node), called the orchestrator or sometimes the master, that is in charge of routing requests to the right node. Distributed